

Politechnika Wrocławska
Sztuczna Inteligencja i Inżynieria Wiedzy

Raport 2

Algorytm Minimax z cięciami alfa-beta
na przykładzie gry Breakthrough

Dawid Pilarski

Numer indeksu: 280468

1. Wprowadzenie

Tematem zadania było zaimplementowanie algorytmu Minimax oraz jego optymalizacji w postaci cięć alfa-beta na przykładzie gry Breakthrough - dwuosobowej gry o sumie zerowej. Idea jest prosta: dwóch graczy na szachownicy, każdy chce się przebić na drugą stronę planszy.

Zadanie składa się z dwóch warstw. Pierwsza to algorytmiczna - Minimax i jego ulepszenie cięciami alfa-beta. Druga to heurystyczna - zestaw funkcji oceny stanu gry, które pozwalają badać jak różne strategie radzą sobie ze sobą. Zaimplementowałem też wersję rozszerzoną, w której każdy gracz może mieć inną heurystykę i inną głębokość.

2. Tło teoretyczne

2.1. Gry dwuosobowe i drzewo gry

Gra dwuosobowa o sumie zerowej to taka, w której zysk jednego gracza jest stratą drugiego. Breakthrough jest jej klasycznym przykładem - albo wygra B, albo wygra W. Z tego typu gier modeluje się drzewo gry: w korzeniu jest stan początkowy, krawędzie to ruchy, liście to stany terminalne. Problem polega na tym, że dla 8x8 Breakthrough drzewo eksploduje wykładniczo - już z pozycji startowej jest 22 możliwych ruchów.

Przeszukanie pełnego drzewa jest niemożliwe, więc stosujemy limit głębokości plus funkcję heurystyczną oceniającą stany niekońcowe.

2.2. Algorytm Minimax

Minimax opiera się na założeniu, że obaj gracze grają optymalnie. Jeden maksymalizuje wartość funkcji oceny, drugi minimalizuje. W rekurencji algorytm na zmianę bierze maksimum lub minimum z wartości dzieci. W liściu zwracamy wartość heurystyki.

Główna wada: Minimax jest „ślepy” - eksploruje całe drzewo do limitu, nawet jeśli pewne gałęzie są oczywiście beznadziejne. Złożoność to $O(b^d)$, gdzie b to średni współczynnik rozgałęzień, a d to głębokość. Przy $d=4$ to setki tysięcy wierzchołków.

2.3. Cięcia alfa-beta

Alfa-beta to optymalizacja, która nie zmienia wyniku, tylko go przyspiesza. Utrzymujemy dwa parametry: α (najlepsza wartość znaleziona dotąd dla MAX) i β (najlepsza dla MIN). Gdy $\alpha \geq \beta$, dalsze przeszukiwanie tej gałęzi nie ma sensu - przeciwnik nigdy nie pozwoli, żeby do niej doszło.

W idealnym przypadku złożoność spada z $O(b^d)$ do $O(b^{d/2})$, czyli można przeszukać dwa razy głębsze drzewo w tym samym czasie. Co ważne - skuteczność cięć zależy od kolejności rozpatrywania ruchów. Im wcześniej znajdziemy dobry ruch, tym więcej gałęzi da się odciąć.

3. Gra Breakthrough

Breakthrough zaprojektował Dan Troyka w 2000 roku. Plansza ma 8x8, każdy gracz zaczyna z 16 pionkami na dwóch skrajnych wierszach. Pionki poruszają się tylko w przód: jedno pole prosto lub po skosie.

Najważniejszy detal mechaniki: bicie odbywa się tylko po skosie. Jeśli pionek przeciwnika stoi na wprost przede mną, nie mogę go zbić - muszę go obejść. To zasadniczo różni Breakthrough od warcabów. Wygrywa gracz, którego pionek pierwszy dotrze do najdalszego wiersza przeciwnika.

4. Implementacja

4.1. Struktura projektu

Całość podzieliłem na pięć plików, każdy z jasną odpowiedzialnością:

- **board.py** - reprezentacja stanu gry. Klasa GameState trzyma planszę, gracza do ruchu i zliczone liczby pionków. Tu jest generacja legalnych ruchów oraz detekcja stanów terminalnych.
- **heuristics.py** - dziewięć heurystyk oceny stanu (5 bazowych + 4 strategie + adaptive).
- **minimax.py** - klasa MinimaxAgent z metodami `_minimax` i `_alphabeta`. Flaga `use_alpha_beta` przełącza tryb.
- **breakthrough.py** - punkt wejścia. Tryb basic (oba agenty z tymi samymi parametrami) i extended (różne parametry per gracz).
- **boards/default.txt** - domyślna plansza startowa 8x8.

4.2. Reprezentacja stanu i generacja ruchów

Plansza to lista list znaków: B (gracz pierwszy), W (gracz drugi), _ (puste pole), o (puste pole, skąd wykonano ostatni ruch). GameState cache'uje liczbę pionków każdego gracza podczas konstrukcji - heurystyki materiałowe nie muszą później przeliczać planszy.

Generacja ruchów (`get_moves`) iteruje po wszystkich pionkach aktualnego gracza i sprawdza trzy kierunki: prosto i dwa skosy. Prosto można iść tylko na puste pole, skosy mogą być na puste lub na pionek przeciwnika (bicie). Bicie na wprost jest zabronione.

4.3. Heurystyki

Zaimplementowałem dziewięć heurystyk - pięć bazowych i cztery strategie złożone:

- **material** - różnica liczby pionków własnych i przeciwnika.
- **advancement** - suma odległości pionków od linii startu.
- **most_advanced** - pozycja najbardziej wysuniętego pionka. Strategia „wyścig”.
- **defensive** - liczba własnych pionków na pierwszych dwóch liniach.
- **mobility** - różnica liczby legalnych ruchów.
- **hybrid** - kombinacja zbalansowana: $5 \cdot \text{material} + 2 \cdot \text{advancement} + 8 \cdot \text{most_advanced} + 3 \cdot \text{defensive}$.
- **aggressive** - silnie premiuje postęp: $2 \cdot \text{material} + 4 \cdot \text{advancement} + 12 \cdot \text{most_advanced}$.

- **defensive_strategy** - silnie premiuje materiał i obronę.
- **adaptive** - zmienia strategię w zależności od fazy gry. W otwarciu (>85% pionków) defensywnie, w końcówce (<40% pionków lub ktoś blisko wygranej) agresywnie, w środkowej grze hybrid.

Funkcja evaluate opakowuje heurystyki: dla stanów terminalnych zwraca $\pm 10^6$, w przeciwnym razie wywołuje wybraną heurystykę.

4.4. Minimax i alfa-beta

Klasa MinimaxAgent ma stałe pole my_player i flagę use_alpha_beta przełączającą wariant. Dzięki temu ten sam agent działa poprawnie z perspektywy obu graczy, co jest kluczowe dla wersji rozszerzonej.

Alfa-beta to ten sam wzorzec rekurencji co Minimax, ale z dodatkowymi parametrami alpha i beta. Po każdej aktualizacji najlepszej wartości sprawdzam $\alpha \geq \beta$ i wychodzę pętlą break - to jest właśnie cięcie.

Cięcia działają lepiej, gdy ruchy są uporządkowane od najbardziej obiecujących. Wprowadziłem proste sortowanie: najpierw bicia (priorytet +100), potem ruchy o większym progresie. Bez tego cięcia działają, ale są mniej skuteczne.

5. Eksperymenty

5.1. Minimax bez cięć - głębokość 2

Punkt odniesienia. Czysty Minimax na małej głębokości - kończy się szybko, ale rozwija dużo węzłów.

```
PS D:\Projekty\breakthrough> python breakthrough.py --default --depth 2 --heuristic hybrid --algorithm minimax
Tryb: basic
  B: heurystyka=hybrid, głębokość=2, algorytm=minimax
  W: heurystyka=hybrid, głębokość=2, algorytm=minimax
  _ _ B B B B B
B _ B B B B B B
B W B _ _ _ _ _
  _ W _ _ _ _ _
W W _ B _ _ _ _
  _ _ _ _ _ _ _
o _ W W W W W W
B _ W W W W W W
Rounds: 27. Winner: B
Nodes visited: 15781 (B: 8141, W: 7640)
Time: 0.233s
```

Algorytm rozwinął ~16 tysięcy węzłów, czas ok. 0.2 sekundy. Bez cięć już tutaj widać, że obciążenie obliczeniowe nie jest trywialne.

5.2. Alfa-beta - głębokość 2

Ta sama głębokość, ten sam stan początkowy, ale z cięciami:

```
PS D:\Projekty\breakthrough> python breakthrough.py --default --depth 2 --heuristic hybrid --algorithm alphabeta
Tryb: basic
  B: heurystyka=hybrid, głębokość=2, algorytm=alphabeta
  W: heurystyka=hybrid, głębokość=2, algorytm=alphabeta
W _ B B B B B B
o _ B B B B B B
  B _ _ _ _ _ _
  B _ W _ _ _ _
  B B _ _ _ _ _
B _ _ _ W _ _ _
W W W _ W W W W
  _ _ W W W W W
Rounds: 30. Winner: W
Nodes visited: 2919 (B: 1459, W: 1460)
Time: 0.042s
```

Liczba węzłów spadła do ~3 tysięcy (ok. 5x mniej), czas również. Ciekawostka: przy $d=2$ może wygrać inny gracz niż w czystym Minimaxie - to efekt sortowania ruchów. Gdy kilka ruchów ma identyczną wartość, kolejność rozpatrywania decyduje o wyborze.

5.3. Minimax bez cięć - głębokość 3

```

PS D:\Projekty\breakthrough> python breakthrough.py --default --depth 3 --heuristic hybrid --algorithm minimax
Tryb: basic
  B: heurystyka=hybrid, głębokość=3, algorytm=minimax
  W: heurystyka=hybrid, głębokość=3, algorytm=minimax
  _ _ _ B B B B
W _ _ _ B B B B
_ B B B _ _ _ _
B _ _ _ _ _ _ _
_ W _ _ _ _ _ _
_ _ _ _ _ _ _ _
o _ W W W W W W
B _ _ _ W W W W
Rounds: 43. Winner: B
Nodes visited: 515438 (B: 265472, W: 249966)
Time: 7.338s

```

Ponad 500 tysięcy odwiedzonych węzłów. Minimax bez cięć na głębokości 3 staje się powoli niewykonalny - głębokość 4 byłaby już nierealistyczna.

5.4. Alfa-beta - głębokość 3

Ta sama głębokość, ale z cięciami:

```

Time: 7.338s
PS D:\Projekty\breakthrough> python breakthrough.py --default --depth 3 --heuristic hybrid --algorithm alphabeta
Tryb: basic
  B: heurystyka=hybrid, głębokość=3, algorytm=alphabeta
  W: heurystyka=hybrid, głębokość=3, algorytm=alphabeta
  _ _ B B B B B B
B _ B B B B B B
_ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _
_ _ _ _ _ W _ _ _
_ O _ _ _ W W W
B W _ W W W W W
Rounds: 33. Winner: B
Nodes visited: 22363 (B: 11557, W: 10806)
Time: 0.320s

```

Ponad 22 tysiące węzłów - 23 razy mniej niż czysty Minimax. Czas spadł z 7 sekund do 0.3s. To zgadza się z teoretyczną granicą $O(b^{d/2})$ - z każdym kolejnym poziomem zysk z cięć rośnie wykładniczo.

5.5. Wersja rozszerzona - dwa różne agenty

Sprawdzam tryb extended - B gra agresywnie, W defensywnie, ta sama głębokość 3:

```

PS D:\Projekty\breakthrough> python breakthrough.py --default --mode extended --heuristic-b aggressive --heuristic-w defensive_strategy --depth-b 3 --depth-w 3
Tryb: extended
  B: heurystyka=aggressive, głębokość=3, algorytm=alphabeta
  W: heurystyka=defensive_strategy, głębokość=3, algorytm=alphabeta
W _ B B _ B B B
o _ _ _ _ _ B
_ B B _ _ _ B _
_ _ _ _ _ _ _ _
_ B _ B B B _ _
_ _ _ _ _ _ _ _
_ W W W W W W W
_ _ _ W W W W W
Rounds: 44. Winner: W
Nodes visited: 47177 (B: 27874, W: 19303)
Time: 0.658s

```

Wersja rozszerzona pozwala na rozegranie partii między dwoma niezależnymi agentami. Pokazuje to też, że Minimax sam w sobie jest deterministyczny, ale różne heurystyki dają różne style gry.

5.6. Heurystyka adaptacyjna

Adaptive zmienia strategię w zależności od fazy gry:

```
PS D:\Projekty\breakthrough> python breakthrough.py --default --depth 3 --heuristic adaptive --algorithm alphabeta
Tryb: basic
B: heurystyka=adaptive, głębokość=3, algorytm=alphabeta
W: heurystyka=adaptive, głębokość=3, algorytm=alphabeta
W _ B B B B B B
o B _ B B B B B
_ B _ _ _ _ _ _
_ _ _ _ _ _ _ _
_ _ _ B _ _ _ _
W _ _ _ _ _ _ _
_ _ W W W W W W
_ _ W W W W W W
Rounds: 28, Winner: W
Nodes visited: 24297 (B: 11682, W: 12615)
Time: 0.401s
```

Adaptive sama ze sobą - można obserwować w stderr, że na początku zachowanie jest defensywne, w środku gry zbalansowane, a w końcówce agresywne. To moim zdaniem najciekawsza heurystyka w pakiecie.

6. Użyte biblioteki

Projekt korzysta wyłącznie z biblioteki standardowej Pythona, bez zewnętrznych zależności:

- **argparse** - parsowanie argumentów linii komend.
- **sys** - operacje wejścia/wyjścia (stdout/stderr zgodnie z treścią zadania).
- **time** - pomiar czasu wykonania algorytmu.
- **copy** - głębokie kopie planszy podczas tworzenia stanów potomnych.

7. Napotkane problemy

- **Limit rund.** Teoretycznie Breakthrough zawsze się kończy, ale dla małych głębokości i pewnych heurystyk gra może trwać bardzo długo. Wprowadziłem limit 500 rund jako zabezpieczenie.
- **Uporządkowanie ruchów.** Bez sortowania alfa-beta nadal działa, ale cięcia są dużo mniej skuteczne. Po wprowadzeniu prostego sortowania (bicia + progres) liczba węzłów spadła zauważalnie.

8. Wnioski

Zysk z cięć alfa-beta jest dramatyczny i rośnie wykładniczo z głębokością. Dla głębokości 3 alfa-beta odwiedza ponad 23 razy mniej węzłów niż czysty Minimax. Dla zadań typu Breakthrough alfa-beta jest praktycznie obowiązkowa - bez niej głębokość 4 staje się niewykonalna w rozsądnym czasie.

Wersja rozszerzona z dwoma niezależnymi agentami jest najciekawszą częścią - dzięki niej można rozegrać dowolne starcia heurystyk i zobaczyć, że przy tej samej głębokości i tym samym algorytmie różne strategie dają wyraźnie różne wyniki. Sam Minimax jest deterministyczny, ale zbiór możliwych „osobowości” agenta tworzony przez różne heurystyki sprawia, że gra robi się ciekawa.

9. Materiały Źródłowe

Pozycje wskazane w treści zadania:

1. Breakthrough (board game) — Wikipedia,
[https://en.wikipedia.org/wiki/Breakthrough_\(board_game\)](https://en.wikipedia.org/wiki/Breakthrough_(board_game)).
2. Breakthrough — opis zasad gry, materiały do listy 2 (zasady rozgrywki, ruchy pionków, warunki zwycięstwa).
3. J.F. Nordstrom, *Introduction to Game Theory: A Discovery Approach*, Whitman College, 2020.
4. S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach* (rozdz. 5 — Adversarial Search), wyd. 4, Pearson, 2020.
5. Materiały wykładowe SIiIW, Politechnika Wrocławska, 2026.
6. Gemini Pro